OSS LAB EXAM

Complete Study Guide — CB3402 Operating Systems & Security

P.S.V College of Engineering and Technology

B.E/B.Tech Practical End Semester Exam | April/May 2026

Second Semester | Regulation 2025 | Time: 3 Hrs | Max: 100 Marks

Answer any ONE question from the list below

Aim/Algorithm	Program	Execution	Viva-Voce	Record	Total
20	30	30	10	10	100

Q1 — UNIX Commands, chmod, setuid, setgid

■ 1. PROGRAM (Simple C Language Version)

Setup / Compile:

```
# First create the test file:
touch testfile.txt
gcc q1.c -o q1
./q1
```

```c
#include <stdio.h>
#include <stdlib.h>
#include <sys/stat.h>

int main() {
    // chmod: change file permissions
    // rwxr-xr-x = 0755
    if (chmod("testfile.txt", 0755) == 0)
        printf("chmod 755 applied successfully\n");
    else
        printf("chmod failed\n");

    // setuid bit = 04000, setgid bit = 02000
    if (chmod("testfile.txt", 04755) == 0)
        printf("setuid bit set successfully\n");

    if (chmod("testfile.txt", 02755) == 0)
        printf("setgid bit set successfully\n");

    printf("\nLinux Permission Basics:\n");
    printf("r=4, w=2, x=1\n");
    printf("chmod 777 = rwxrwxrwx\n");
    printf("chmod 755 = rwxr-xr-x\n");
    printf("chmod 644 = rw-r--r--\n");
    return 0;
}
```

■ 2. SAMPLE OUTPUT

```
chmod 755 applied successfully
setuid bit set successfully
setgid bit set successfully

Linux Permission Basics:
r=4, w=2, x=1
chmod 777 = rwxrwxrwx
chmod 755 = rwxr-xr-x
chmod 644 = rw-r--r--
```

■ 3. PROGRAM LOGIC (3-5 Points)

- chmod() system call changes file permissions.
- Permission format: Owner | Group | Others (each: r=4,w=2,x=1).
- setuid (04000): Run as file owner, not the current user.
- setgid (02000): Run with group privileges of file owner.
- Combine: chmod("file", 04755) sets setuid + rwxr-xr-x.

■ 4. VIVA QUESTIONS & ANSWERS

Q1. What is chmod?	➜ chmod changes file permissions. Syntax: chmod 755 file.
Q2. What does 755 mean?	➜ Owner=rwx(7), Group=r-x(5), Others=r-x(5).
Q3. What is setuid?	➜ When set, program runs with owner's privileges, not the user who runs it.
Q4. What is setgid?	➜ When set, program runs with group's privileges of the file owner.
Q5. What is sticky bit?	➜ Prevents users from deleting others' files in shared directories. Set with chmod +t.

■ 5. MEMORY TRICK

■ ***Remember: r=4, w=2, x=1. Add them up! 7=rwx, 6=rw-, 5=r-x, 4=r--. setUID=4xxx, setGID=2xxx.***

Q2 — UNIX System Calls: fork, exec, wait, getpid, exit, stat, opendir, readdir

■ 1. PROGRAM (Simple C Language Version)

Setup / Compile:

```
gcc q2.c -o q2
./q2
```

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <sys/stat.h>
#include <dirent.h>

int main() {
    pid_t pid;
    int status;
    struct stat st;
    DIR *dir;
    struct dirent *entry;

    // getpid - get process ID
    printf("Parent PID: %d\n", getpid());

    // fork - create child process
    pid = fork();

    if (pid < 0) {
        printf("Fork failed!\n");
        exit(1);
    }
    else if (pid == 0) {
        // CHILD PROCESS
        printf("Child PID: %d\n", getpid());
        printf("Child: running ls using exec\n");
        execlp("ls", "ls", "-l", NULL);  // exec replaces process
        exit(0);
    }
    else {
        // PARENT PROCESS
        wait(&status);  // wait for child to finish
        printf("Parent: child finished\n");

        // stat - file information
        stat(".", &st);
        printf("Dir size: %ld bytes\n", st.st_size);

        // opendir + readdir - list directory
        dir = opendir(".");
        printf("Files in current dir:\n");
        while ((entry = readdir(dir)) != NULL)
            printf("  %s\n", entry->d_name);
        closedir(dir);
    }
    return 0;
}
```

■ 2. SAMPLE OUTPUT

```
Parent PID: 1234
Child PID: 1235
Child: running ls using exec
total 12
-rwxr-xr-x 1 user user 8472 Jun  4 q2
-rw-r--r-- 1 user user  890 Jun  4 q2.c
Parent: child finished
Dir size: 4096 bytes
Files in current dir:
  .
  ..
  q2
  q2.c
```

■ 3. PROGRAM LOGIC (3-5 Points)

- fork() creates a copy of current process. Returns 0 to child, child-PID to parent.
- exec() replaces child process with a new program (ls here).
- wait() makes parent wait until child finishes.
- getpid() returns the process ID of the current process.
- opendir()/readdir() open and read directory entries one by one.

■ 4. VIVA QUESTIONS & ANSWERS

Q1. What does fork() return?	➜ Returns 0 to child process, child's PID to parent, -1 on error.
Q2. Difference between fork and exec?	➜ fork creates a NEW process. exec REPLACES current process with another program.
Q3. What is a zombie process?	➜ A child that has finished but parent hasn't called wait() yet.
Q4. What is an orphan process?	➜ A child whose parent has died. It is adopted by init (PID 1).
Q5. What does wait() do?	➜ Parent waits for child to finish. Prevents zombie processes.

■ 5. MEMORY TRICK

■ ***FORK creates child (returns 0 to child). EXEC replaces. WAIT for child. GET-PID gets your ID. Remember: F-E-W-G.***

Q3 — CPU Scheduling Algorithms

■ 1. PROGRAM (Simple C Language Version)

Setup / Compile:

```
gcc q3.c -o q3
./q3
```

```
/* FCFS CPU Scheduling - Simplest to write! */
#include <stdio.h>

int main() {
    int n, i, j;
    printf("Enter number of processes: ");
    scanf("%d", &n);

    int pid[10], bt[10], wt[10], tat[10];
    float avgwt = 0, avgtat = 0;

    for (i = 0; i < n; i++) {
        pid[i] = i + 1;
        printf("Enter burst time for P%d: ", i+1);
        scanf("%d", &bt[i]);
    }

    // FCFS: processes run in arrival order
    wt[0] = 0;  // first process waits 0
    for (i = 1; i < n; i++)
        wt[i] = wt[i-1] + bt[i-1];

    printf("\nPID  BT  WT  TAT\n");
    for (i = 0; i < n; i++) {
        tat[i] = wt[i] + bt[i];
        avgwt  += wt[i];
        avgtat += tat[i];
        printf("P%d   %d   %d   %d\n",
               pid[i], bt[i], wt[i], tat[i]);
    }

    printf("\nAvg Waiting Time    : %.2f\n", avgwt/n);
    printf("Avg Turnaround Time : %.2f\n", avgtat/n);
    return 0;
}
```

■ 2. SAMPLE OUTPUT

```
Enter number of processes: 3
Enter burst time for P1: 5
Enter burst time for P2: 3
Enter burst time for P3: 8

PID  BT  WT  TAT
P1   5   0   5
P2   3   5   8
P3   8   8   16

Avg Waiting Time    : 4.33
Avg Turnaround Time : 9.67
```

■ 3. PROGRAM LOGIC (3-5 Points)

- FCFS = First Come First Served. Processes run in the order they arrive.
- Waiting Time (WT) of P[i] = WT[i-1] + BT[i-1]. First process WT = 0.
- Turnaround Time (TAT) = Waiting Time + Burst Time.
- Average WT and TAT = sum / number of processes.
- FCFS is non-preemptive — once started, a process runs to completion.

■ 4. VIVA QUESTIONS & ANSWERS

Q1. What is Burst Time?	➜ The time a process needs the CPU to complete execution.
Q2. What is Waiting Time?	➜ Time a process spends waiting in the ready queue before getting CPU.
Q3. What is Turnaround Time?	➜ Total time from process arrival to completion. TAT = WT + BT.
Q4. What is FCFS?	➜ First Come First Served. Processes get CPU in arrival order. Simple but may cause convoy effect.
Q5. What is the convoy effect?	➜ Short processes wait behind a long process in FCFS, causing poor performance.

■ 5. MEMORY TRICK

■ ***WT[0]=0, WT[i] = WT[i-1] + BT[i-1]. TAT = WT + BT. Just add previous burst to get next wait!***

Q4 — Implementation of Semaphores

■ 1. PROGRAM (Simple C Language Version)

Setup / Compile:

```
gcc q4.c -o q4 -lpthread
./q4
```

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>

sem_t sem;   // semaphore variable
int shared = 0;  // shared resource

void *producer(void *arg) {
    sem_wait(&sem);   // P() - decrement, lock
    shared++;
    printf("Producer: shared = %d\n", shared);
    sem_post(&sem);   // V() - increment, unlock
    return NULL;
}

void *consumer(void *arg) {
    sem_wait(&sem);   // P() - wait for access
    shared--;
    printf("Consumer: shared = %d\n", shared);
    sem_post(&sem);   // V() - release
    return NULL;
}

int main() {
    pthread_t t1, t2;

    sem_init(&sem, 0, 1);  // init semaphore to 1 (binary)

    pthread_create(&t1, NULL, producer, NULL);
    pthread_create(&t2, NULL, consumer, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    sem_destroy(&sem);
    printf("Final shared value: %d\n", shared);
    return 0;
}
```

■ 2. SAMPLE OUTPUT

```
Producer: shared = 1
Consumer: shared = 0
Final shared value: 0
```

■ 3. PROGRAM LOGIC (3-5 Points)

- Semaphore is a variable used to control access to shared resources.
- sem_wait() (P operation): Decrements semaphore. If 0, the thread blocks/waits.
- sem_post() (V operation): Increments semaphore. Wakes up waiting thread.

• Binary semaphore (value 0 or 1) works like a mutex lock.

• sem_init(&sem;, 0, 1) — 0=process-local, 1=initial value (unlocked).

■ 4. VIVA QUESTIONS & ANSWERS

Q1. What is a semaphore?	➜ A synchronization variable that controls access to shared resources. It's an integer with P and V operations.
Q2. What is P operation?	➜ P (wait/down): Decrements the semaphore. If value is 0, process blocks.
Q3. What is V operation?	➜ V (signal/up): Increments the semaphore. Wakes up blocked processes.
Q4. Difference binary vs counting semaphore?	➜ Binary: only 0 or 1 (like a lock). Counting: can have any non-negative value.
Q5. What is mutex?	➜ Mutual Exclusion. Binary semaphore used to allow only one process in critical section.

■ 5. MEMORY TRICK

■ ***SEM = Guard. WAIT = Lock the door (P). POST = Open the door (V). Compile with -lpthread always!***

Q5 — Implementation of Shared Memory

■ 1. PROGRAM (Simple C Language Version)

Setup / Compile:

```
gcc q5.c -o q5
./q5
```

```c
#include <stdio.h>
#include <stdlib.h>
#include <sys/ipc.h>
#include <sys/shm.h>
#include <string.h>

int main() {
    int shmid;
    char *shmptr;
    key_t key = 1234;  // unique key

    // Create shared memory segment (1024 bytes)
    shmid = shmget(key, 1024, IPC_CREAT | 0666);
    if (shmid < 0) {
        printf("shmget failed\n");
        exit(1);
    }
    printf("Shared memory created. ID: %d\n", shmid);

    // Attach shared memory to process address space
    shmptr = (char *) shmat(shmid, NULL, 0);
    if (shmptr == (char *) -1) {
        printf("shmat failed\n");
        exit(1);
    }

    // Write to shared memory
    strcpy(shmptr, "Hello from Shared Memory!");
    printf("Written: %s\n", shmptr);

    // Read from shared memory
    printf("Read back: %s\n", shmptr);

    // Detach shared memory
    shmdt(shmptr);
    printf("Shared memory detached\n");

    // Remove shared memory
    shmctl(shmid, IPC_RMID, NULL);
    printf("Shared memory removed\n");

    return 0;
}
```

■ 2. SAMPLE OUTPUT

```
Shared memory created. ID: 12345
Written: Hello from Shared Memory!
Read back: Hello from Shared Memory!
Shared memory detached
Shared memory removed
```

■ 3. PROGRAM LOGIC (3-5 Points)

- Shared memory is the FASTEST IPC method — processes share the same memory region.
- shmget() creates/gets a shared memory segment using a key.
- shmat() attaches (maps) the shared memory to the process address space.
- shmdt() detaches shared memory from process (does NOT delete it).
- shmctl(IPC_RMID) permanently removes the shared memory segment.

■ 4. VIVA QUESTIONS & ANSWERS

Q1. What is shared memory?	➜ A region of memory that can be accessed by multiple processes simultaneously. Fastest IPC method.
Q2. What is shmget()?	➜ Creates or accesses a shared memory segment. Returns a shared memory ID.
Q3. What is shmat()?	➜ Attaches shared memory to the calling process's address space.
Q4. What is shmdt()?	➜ Detaches shared memory from process but does NOT delete it.
Q5. Difference shmdt vs shmctl?	➜ shmdt detaches from process. shmctl with IPC_RMID deletes the segment permanently.

■ 5. MEMORY TRICK

■ 4 steps: GET → ATTACH → USE → DETACH → REMOVE. shmGET, shmAT, shmDT, shmCTL. Like renting a room!

Q6 — Banker's Algorithm (Deadlock Detection & Avoidance)

■ 1. PROGRAM (Simple C Language Version)

Setup / Compile:

```
gcc q6.c -o q6
./q6
```

```c
#include <stdio.h>

#define P 3  // processes
#define R 3  // resources

int main() {
    int alloc[P][R] = {{0,1,0},{2,0,0},{3,0,2}};
    int max[P][R]   = {{7,5,3},{3,2,2},{9,0,2}};
    int avail[R]    = {3,3,2};
    int need[P][R], finish[P]={0}, safeseq[P];
    int i, j, k=0, found;
    int work[R];

    // Calculate need = max - allocation
    for (i=0;i<P;i++)
        for (j=0;j<R;j++)
            need[i][j] = max[i][j] - alloc[i][j];

    for (j=0;j<R;j++) work[j]=avail[j];

    printf("Need Matrix:\n");
    for(i=0;i<P;i++){
        printf("P%d: ",i);
        for(j=0;j<R;j++) printf("%d ",need[i][j]);
        printf("\n");
    }

    // Find safe sequence
    int count = 0;
    while (count < P) {
        found = 0;
        for (i=0;i<P;i++) {
            if (!finish[i]) {
                int ok=1;
                for(j=0;j<R;j++)
                    if(need[i][j]>work[j]){ok=0;break;}
                if (ok) {
                    for(j=0;j<R;j++) work[j]+=alloc[i][j];
                    safeseq[k++]=i;
                    finish[i]=1;
                    found=1; count++;
                }
            }
        }
        if (!found) { printf("UNSAFE STATE - DEADLOCK!\n"); return 0; }
    }

    printf("\nSAFE STATE!\nSafe Sequence: ");
    for(i=0;i<P;i++) printf("P%d ",safeseq[i]);
    printf("\n");
    return 0;
}
```

■ 2. SAMPLE OUTPUT

```
Need Matrix:
P0: 7 4 3
P1: 1 2 2
P2: 6 0 0

SAFE STATE!
Safe Sequence: P1 P2 P0
```

■ 3. PROGRAM LOGIC (3-5 Points)

- Banker's Algorithm avoids deadlock by checking if resource allocation leads to SAFE state.
- Need = Max - Allocation (what each process still needs).
- Work = Available resources. Find a process where Need <= Work.
- If found: add its allocation to Work, mark it finished (simulating completion).
- If all processes finish = SAFE state. Otherwise = UNSAFE (possible deadlock).

■ 4. VIVA QUESTIONS & ANSWERS

Q1. What is deadlock?	➜ A state where processes wait for each other's resources forever and no one can proceed.
Q2. What is Banker's Algorithm?	➜ Deadlock avoidance algorithm. It grants resources only if the system stays in a safe state.
Q3. What is safe state?	➜ A state where there exists at least one safe sequence to complete all processes.
Q4. What is Need matrix?	➜ Need[i][j] = Max[i][j] - Allocation[i][j]. What process i still needs of resource j.
Q5. What are 4 conditions for deadlock?	➜ Mutual Exclusion, Hold & Wait, No Preemption, Circular Wait.

■ 5. MEMORY TRICK

■ ***NEED = MAX - ALLOC. Check NEED <= WORK. If yes: add ALLOC to WORK, finish process. Loop until all done = SAFE!***

Q7 — Memory Allocation: First Fit, Best Fit, Worst Fit

■ 1. PROGRAM (Simple C Language Version)

Setup / Compile:

```
gcc q7.c -o q7
./q7
```

```
#include <stdio.h>
#include <string.h>

#define MAX 10

void firstFit(int blocks[], int nb, int proc[], int np) {
    int alloc[MAX]; int i, j;
    memset(alloc, -1, sizeof(alloc));
    for(i=0;i<np;i++)
        for(j=0;j<nb;j++)
            if(blocks[j]>=proc[i]){alloc[i]=j;blocks[j]-=proc[i];break;}
    printf("\n--- FIRST FIT ---\n");
    printf("Process\tSize\tBlock\n");
    for(i=0;i<np;i++)
        printf("P%d\t%d\t%s\n",i+1,proc[i],alloc[i]==-1?"Not Alloc":"Alloc");
}

void bestFit(int blocks[], int nb, int proc[], int np) {
    int alloc[MAX]; int i, j, best;
    memset(alloc, -1, sizeof(alloc));
    for(i=0;i<np;i++){
        best=-1;
        for(j=0;j<nb;j++)
            if(blocks[j]>=proc[i]&&(best==-1||blocks[j]<blocks[best]))
                best=j;
        if(best!=-1){alloc[i]=best;blocks[best]-=proc[i];}
    }
    printf("\n--- BEST FIT ---\n");
    printf("Process\tSize\tBlock\n");
    for(i=0;i<np;i++)
        printf("P%d\t%d\t%s\n",i+1,proc[i],alloc[i]==-1?"Not Alloc":"Alloc");
}

void worstFit(int blocks[], int nb, int proc[], int np) {
    int alloc[MAX]; int i, j, worst;
    memset(alloc, -1, sizeof(alloc));
    for(i=0;i<np;i++){
        worst=-1;
        for(j=0;j<nb;j++)
            if(blocks[j]>=proc[i]&&(worst==-1||blocks[j]>blocks[worst]))
                worst=j;
        if(worst!=-1){alloc[i]=worst;blocks[worst]-=proc[i];}
    }
    printf("\n--- WORST FIT ---\n");
    printf("Process\tSize\tBlock\n");
    for(i=0;i<np;i++)
        printf("P%d\t%d\t%s\n",i+1,proc[i],alloc[i]==-1?"Not Alloc":"Alloc");
}

int main() {
    int b1[]={100,500,200,300,600};
    int b2[]={100,500,200,300,600};
    int b3[]={100,500,200,300,600};
    int proc[]={212,417,112,426};
    firstFit(b1,5,proc,4);
    bestFit(b2,5,proc,4);
    worstFit(b3,5,proc,4);
    return 0;
}
```

■ 2. SAMPLE OUTPUT

```
--- FIRST FIT ---
Process  Size  Block
P1       212   Alloc
P2       417   Alloc
P3       112   Alloc
P4       426   Not Alloc

--- BEST FIT ---
Process  Size  Block
P1       212   Alloc
P2       417   Alloc
P3       112   Alloc
P4       426   Not Alloc

--- WORST FIT ---
Process  Size  Block
P1       212   Alloc
P2       417   Alloc
P3       112   Alloc
P4       426   Not Alloc
```

■ 3. PROGRAM LOGIC (3-5 Points)

- First Fit: Scan from beginning, allocate to FIRST block that fits. Fast but fragmented.
- Best Fit: Find the SMALLEST block that fits. Least wasted space inside block.
- Worst Fit: Find the LARGEST block that fits. Leaves largest possible remaining hole.
- After allocating, reduce block size by process size.
- Fixed partition: Memory divided into fixed-size blocks that don't change.

■ 4. VIVA QUESTIONS & ANSWERS

Q1. What is First Fit?	➜ Allocates memory in the FIRST hole that is big enough. Fastest but causes external fragmentation.
Q2. What is Best Fit?	➜ Finds the SMALLEST hole that fits the process. Minimizes wasted space in that block.
Q3. What is Worst Fit?	➜ Finds the LARGEST hole. Leaves the largest remaining space for future use.
Q4. What is fragmentation?	➜ Internal: wasted space inside allocated block. External: free space scattered in small pieces.
Q5. Which is best algorithm?	➜ Best Fit reduces fragmentation. First Fit is fastest. Worst Fit leaves large holes for big processes.

■ 5. MEMORY TRICK

■ ***First=FIRST fit. Best=SMALLEST fit. Worst=LARGEST fit. Think: Good friend gives BEST piece of cake (smallest slice he keeps)!***

Q8 — Page Replacement: FIFO, LRU, LFU

■ 1. PROGRAM (Simple C Language Version)

Setup / Compile:

```
gcc q8.c -o q8
./q8
```

```c
#include <stdio.h>
#include <string.h>

#define FRAMES 3
#define PAGES  7

int inFrames(int frames[], int n, int page) {
    int i;
    for(i=0;i<n;i++) if(frames[i]==page) return 1;
    return 0;
}

void fifo(int ref[], int n) {
    int frames[FRAMES], ptr=0, faults=0, i, j;
    for(i=0;i<FRAMES;i++) frames[i]=-1;
    printf("\n--- FIFO ---\n");
    for(i=0;i<n;i++) {
        if(!inFrames(frames,FRAMES,ref[i])) {
            frames[ptr]=ref[i]; ptr=(ptr+1)%FRAMES; faults++;
            printf("Page %d -> FAULT ",ref[i]);
        } else printf("Page %d -> HIT   ",ref[i]);
        for(j=0;j<FRAMES;j++) printf("%d ",frames[j]);
        printf("\n");
    }
    printf("Total Page Faults (FIFO): %d\n",faults);
}

void lru(int ref[], int n) {
    int frames[FRAMES], used[FRAMES], faults=0, i, j, lru_idx;
    for(i=0;i<FRAMES;i++){frames[i]=-1; used[i]=0;}
    printf("\n--- LRU ---\n");
    for(i=0;i<n;i++) {
        if(!inFrames(frames,FRAMES,ref[i])) {
            lru_idx=0;
            for(j=1;j<FRAMES;j++) if(used[j]<used[lru_idx]) lru_idx=j;
            frames[lru_idx]=ref[i]; used[lru_idx]=i; faults++;
            printf("Page %d -> FAULT ",ref[i]);
        } else {
            for(j=0;j<FRAMES;j++) if(frames[j]==ref[i]) used[j]=i;
            printf("Page %d -> HIT   ",ref[i]);
        }
        for(j=0;j<FRAMES;j++) printf("%d ",frames[j]);
        printf("\n");
    }
    printf("Total Page Faults (LRU): %d\n",faults);
}

int main() {
    int ref[] = {1,2,3,4,2,1,5};
    fifo(ref, PAGES);
    lru(ref, PAGES);
    printf("\n(LFU: replace page with lowest use count)\n");
    return 0;
}
```

■ 2. SAMPLE OUTPUT

```
--- FIFO ---
Page 1 -> FAULT 1 -1 -1
Page 2 -> FAULT 1 2 -1
Page 3 -> FAULT 1 2 3
Page 4 -> FAULT 4 2 3
Page 2 -> HIT   4 2 3
Page 1 -> FAULT 4 1 3
Page 5 -> FAULT 4 1 5
Total Page Faults (FIFO): 6

--- LRU ---
Page 1 -> FAULT ...
Total Page Faults (LRU): 5
```

■ 3. PROGRAM LOGIC (3-5 Points)

- Page replacement happens when a needed page is NOT in memory (page fault).
- FIFO: Replace the page that came in FIRST (oldest page). Use circular pointer.
- LRU: Replace the page that was NOT USED for the LONGEST time (least recently used).
- LFU: Replace the page used LEAST NUMBER OF TIMES (lowest frequency count).
- Goal: Minimize page faults to improve memory performance.

■ 4. VIVA QUESTIONS & ANSWERS

Q1. What is a page fault?	➜ When a required page is not in memory (RAM), OS must load it from disk.
Q2. What is FIFO replacement?	➜ Replace the oldest page in memory (first one that came in).
Q3. What is LRU?	➜ Least Recently Used. Replace the page not used for the longest time.
Q4. What is Belady's Anomaly?	➜ In FIFO, increasing number of frames can sometimes INCREASE page faults. Does not happen in LRU.
Q5. What is thrashing?	➜ When system spends more time on page swapping than actual execution due to too many page faults.

■ 5. MEMORY TRICK

■ ***FIFO=FIRST in, FIRST out (queue). LRU=LEAST Recently Used (look BACK in time). LFU=LEAST Frequently Used (count hits). F-R-F!***

■ IMPORTANT QUESTIONS FIRST — Study Priority

■■■ MUST STUDY FIRST

Q4 — Semaphores (Very frequently asked, simple code)

Q5 — Shared Memory (Short and important IPC)

Q2 — fork/exec/wait (Core UNIX topic)

Q3 — FCFS Scheduling (Easiest scheduling, always comes)

■■ STUDY SECOND

Q7 — Memory Allocation (First/Best/Worst Fit - scoring program)

Q8 — Page Replacement (FIFO is very easy to write)

Q6 — Banker's Algorithm (Conceptually important)

■ STUDY IF TIME PERMITS

Q1 — chmod/permissions (More theoretical, Linux commands)

■ EASY PROGRAMS — Score Maximum Marks

■ EASIEST: Q5 (Shared Memory) — 4 functions, write & read, done!

■ EASY: Q4 (Semaphores) — sem_init, sem_wait, sem_post, 3 functions.

■ EASY: Q3 (FCFS) — Just add up burst times, simple formula.

■ EASY: Q8 FIFO — Circular pointer logic, straightforward.

■ EASY: Q7 First Fit — Find first block >= process size.

■ FREQUENTLY ASKED VIVA QUESTIONS

Q1. What is IPC?	➜ Inter-Process Communication. Methods: Shared Memory, Pipes, Message Queues, Semaphores.
Q2. What is a process?	➜ A program in execution with its own PCB, memory, and resources.
Q3. What is a thread?	➜ Lightweight process. Shares memory with parent but has own stack and registers.
Q4. What is critical section?	➜ Code section that accesses shared resources. Only one process should execute it at a time.
Q5. What is deadlock?	➜ All processes blocked waiting for resources held by each other. 4 conditions: ME, HW, NP, CW.
Q6. What is paging?	➜ Memory management: divide logical memory into pages, physical memory into frames of equal size.

Q7. What is segmentation?	➜ Divide memory into variable-sized segments based on logical units (code, data, stack).
Q8. What is virtual memory?	➜ Technique to run programs larger than physical RAM using disk as extension of RAM.
Q9. What is scheduling?	➜ OS decides which process gets CPU next. Algorithms: FCFS, SJF, Round Robin, Priority.
Q10. What is a semaphore?	➜ Integer variable for synchronization. P() decrements (wait), V() increments (signal).
Q11. What is mutual exclusion?	➜ Only ONE process at a time can be in the critical section.
Q12. What is context switching?	➜ Saving state of current process and loading state of next process when switching CPU.

■■ LAST 10-MINUTE REVISION — One Line Summary

Question	One-Line Summary
Q1 — chmod/setuid/setgid	chmod() changes permissions; r=4,w=2,x=1; setuid=04xxx runs as owner.
Q2 — fork/exec/wait	fork() creates child; exec() replaces process; wait() waits for child to finish.
Q3 — FCFS Scheduling	WT[i] = WT[i-1] + BT[i-1]; TAT = WT + BT; Average = Sum/N.
Q4 — Semaphores	sem_init; sem_wait()=P=lock; sem_post()=V=unlock; compile with -lpthread.
Q5 — Shared Memory	shmget() create; shmat() attach; use it; shmdt() detach; shmctl(IPC_RMID) delete.
Q6 — Banker's Algorithm	Need=Max-Alloc; find process where Need<=Work; add Alloc to Work; safe if all finish.
Q7 — Memory Allocation	First=first fit; Best=smallest fit; Worst=largest fit; reduce block after allocating.
Q8 — Page Replacement	FIFO=oldest out; LRU=least recently used out; LFU=least frequently used out.

■ Best of luck for your exam! You've got this! — CB3402 OSS Lab | April/May 2026